API REFERENCE MANUAL

Rafiq

Healthcare Management System

API VERSION	v1.0
AUTHORS	ITI Graduation Project Team
UNIVERSITY	Information Technology Institute (ITI)
ACADEMIC YEAR	2025 – 2026
DOCUMENT DATE	August 2026
BASE URL	`https://your-domain.com`

Table of Contents

1. Introduction
2. Authentication
3. Account Management
4. Patient Profiles
5. Emergency Contacts
6. Medical Records — Lab Reports
7. Medical Records — Imaging Reports
8. Medical Records — General Documents
9. Medical Report PDF
10. Prescriptions
11. User Medicines
12. Medication Reminder Schedules
13. Medication Reminder Engine
14. Appointments
15. AI Chat
16. Voice Agent
17. Speech Services
18. Notifications
19. Reviews
20. Administration
21. Data Models Reference
22. Error Reference
23. Authorization Matrix
24. Request Lifecycle
25. Best Practices
26. Glossary

1. Introduction

1.1 Overview

Rafiq is a comprehensive healthcare management platform designed to help individuals and families securely store, manage, and understand their complete medical history. The platform integrates AI-powered document analysis, intelligent medication reminders, appointment tracking, and a conversational health assistant — all behind a single, well-structured REST API.

This document serves as the authoritative reference for every endpoint exposed by the Rafiq backend. It is intended for frontend developers, mobile engineers, and integration partners who need to understand the exact request shape, response structure, validation rules, and business behavior of each operation.

1.2 Base URL

```
https://your-domain.com
```

All endpoint paths in this document are relative to this base URL. For example, `POST /api/auth/register` maps to `https://your-domain.com/api/auth/register`.

1.3 API Versioning

The administration endpoints use an explicit version prefix (`/api/v1/admin/...`). All other endpoints are currently unversioned. Breaking changes will be introduced via a new version prefix.

1.4 Content Types

Direction	Content-Type
Standard JSON request body	`application/json`
File upload request body	`multipart/form-data`
Standard JSON response	`application/json`
PDF report download	`application/pdf`
Audio TTS response	`audio/mpeg`

1.5 Date & Time Format

All timestamps are in **ISO 8601 UTC** format unless otherwise noted:

```
2026-08-04T18:30:00Z
```

Dates without time (e.g., date of birth, report date) use `YYYY-MM-DD`. Times without dates (e.g., reminder times) use the `TimeSpan` format `HH:mm:ss`.

1.6 Standard Response Envelope

Every JSON response from the API wraps its payload in a consistent envelope:

```
{
  "success": true,
  "message": "OK",
  "data": { ... },
  "errors": [],
  "errorCode": null
}
```

Field	Type	Description
`success`	`boolean`	`true` when the operation completed without errors
`message`	`string`	A human-readable status description
`data`	`object | null`	The operation's payload; `null` on failure
`errors`	`string[]`	Validation or business-rule error messages
`errorCode`	`string | null`	A machine-readable error code for programmatic handling

1.7 HTTP Status Codes

Code	Meaning	When Used
`200 OK`	Success	GET, PUT, PATCH, DELETE operations
`201 Created`	Resource created	POST operations that create a new entity
`202 Accepted`	Accepted for async processing	Background AI operations
`400 Bad Request`	Validation or business rule failure	Invalid input or state conflict
`401 Unauthorized`	Missing or invalid JWT	Protected endpoints accessed without a valid token
`403 Forbidden`	Insufficient role/permission	User authenticated but not authorized for this resource
`404 Not Found`	Resource does not exist	ID references a missing record
`409 Conflict`	Duplicate or conflicting state	Registering an existing email
`422 Unprocessable Entity`	Semantic validation failure	Structural input issues
`500 Internal Server Error`	Unexpected server failure	Unhandled exceptions

2. Authentication

The Rafiq API uses **JWT Bearer** tokens for authentication. After login or registration, the server returns a short-lived `accessToken` and a long-lived `refreshToken`. The client must include the access token in the `Authorization` header of every protected request.

Authorization Header

```
Authorization: Bearer <accessToken>
```

Token Lifetimes

Token	Typical Lifetime
Access Token	60 minutes
Refresh Token	7 days

When the access token expires, exchange the refresh token for a new pair using `POST /api/auth/refresh-token`. Refresh tokens are rotated on each use (one-time use) and are revocable.

2.1 Register

`POST /api/auth/register` — **`multipart/form-data`**

Creates a new user account. The endpoint accepts a profile image as an optional file upload alongside the registration fields. On success, the user account is created with the `User` role and requires phone number verification before full access is granted.

Authorization: None (public endpoint)

Request Headers:

Header	Value
`Content-Type`	`multipart/form-data`

Request Body (Form Fields):

Field	Type	Required	Description
`firstName`	`string`		User's given name. Max 100 characters
`lastName`	`string`		User's family name. Max 100 characters
`email`	`string`		Valid email address. Max 256 characters
`phoneNumber`	`string`		Egyptian mobile number matching `^01[0125][0-9]{8}$`
`password`	`string`		Min 8 characters; must contain uppercase, lowercase, digit, and special character
`confirmPassword`	`string`		Must exactly match `password`
`profileImage`	`file`		JPEG, PNG, WEBP, or GIF; max 5 MB

Validation Rules: - Phone numbers must be valid Egyptian mobile numbers (Vodafone `010`, Orange `012`, Etisalat `011`, WE `015`). - Password complexity: at least one uppercase letter, one lowercase letter, one digit, and one special character. - Profile images must be one of: `image/jpeg`, `image/png`, `image/webp`, `image/gif` and must not exceed 5 MB. - The maximum request size is 6 MB.

Example Request:

```
POST /api/auth/register
Content-Type: multipart/form-data

firstName=Ahmed
lastName=Hassan
email=ahmed@example.com
phoneNumber=01012345678
password=MyPass1!
confirmPassword=MyPass1!
```

Success Response — `201 Created` :

```
{
  "success": true,
  "message": "OK",
  "data": {
    "userId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
    "email": "ahmed@example.com",
    "phoneNumber": "01012345678",
    "role": "User",
    "requiresEmailVerification": false,
    "profileImageUrl": null
  }
}
```


Response Fields:

Field	Type	Description
`userId`	`guid`	The newly created user's unique identifier
`email`	`string`	Registered email address
`phoneNumber`	`string`	Registered phone number
`role`	`string`	Assigned role — always `"User"` on registration
`requiresEmailVerification`	`boolean`	Whether email confirmation is required
`profileImageUrl`	`string | null`	URL of the uploaded profile image

Error Responses:

Status	Condition
`400`	Validation failure (weak password, invalid email format, phone format mismatch)
`409`	An account with this email already exists

2.2 Login

> **`POST /api/auth/login`** — **`application/json`**

Authenticates a user with their email or phone number and password. Returns a JWT access token and refresh token pair.

Authorization: None (public endpoint)

Request Body:

```
{
  "loginIdentifier": "ahmed@example.com",
  "password": "MyPass1!"
}
```

Field	Type	Required	Description
`loginIdentifier`	`string`		Email address or phone number
`password`	`string`		Account password

Success Response — `200 OK` :

```
{
  "success": true,
  "message": "Login successful.",
  "data": {
    "accessToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
    "refreshToken": "dGhpcyBpcyBhIHJlZnJlc2ggdG9rZW4...",
    "accessTokenExpiresAt": "2026-08-04T19:30:00Z",
    "refreshTokenExpiresAt": "2026-08-11T18:30:00Z",
    "hasEmergencyContacts": true,
    "profileImageUrl": "https://cdn.example.com/avatars/ahmed.jpg"
  }
}
```

Field	Type	Description
`accessToken`	`string`	JWT Bearer token; include in all subsequent protected requests
`refreshToken`	`string`	Used to obtain a new access token when the current one expires
`accessTokenExpiresAt`	`datetime`	UTC expiry timestamp for the access token
`refreshTokenExpiresAt`	`datetime`	UTC expiry timestamp for the refresh token
`hasEmergencyContacts`	`boolean`	Indicates whether the user has configured emergency contacts
`profileImageUrl`	`string | null`	Profile picture URL for display in the app shell

Error Responses:

Status	Condition
`400`	Invalid credentials or account does not exist
`401`	Account is deactivated by an administrator

2.3 Login with Google

> **`POST /api/auth/google`** — **`application/json`**

Authenticates a user via Google Sign-In. The client must obtain a Google ID Token on the frontend using the Google Identity Services SDK, then send it to this endpoint. If the user does not yet exist in the system, a new account is automatically created.

Authorization: None (public endpoint)

Request Body:

```
{
  "idToken": "eyJhbGciOiJSUzI1NiIsImtpZCI6..."
}
```

Field	Type	Required	Description
`idToken`	`string`		Google ID Token obtained from Google Identity Services

Success Response — `200 OK` : Same structure as Login (`AuthResponseDto`).

2.4 Logout

> **`POST /api/auth/logout`** — **`application/json`**

Invalidates the current refresh token, effectively ending the session. The access token remains technically valid until its natural expiry, so client-side logout (clearing local storage) should accompany this call.

Authorization: Required

Request Body:

```
{
  "refreshToken": "dGhpcyBpcyBhIHJlZnJlc2ggdG9rZW4..."
}
```

Success Response — `200 OK` :

```
{
  "success": true,
  "message": "Logged out successfully.",
  "data": null
}
```

2.5 Refresh Token

> `POST /api/auth/refresh-token` — `application/json`

Exchanges a valid refresh token for a new access token and refresh token pair. The old refresh token is immediately invalidated after use (token rotation). Device information and IP address are automatically captured from the request context.

Authorization: None (uses refresh token in body)

Request Body:

```
{
  "refreshToken": "dGhpcyBpcyBhIHJlZnJlc2ggdG9rZW4..."
}
```

Success Response — `200 OK` : Same structure as Login (`AuthResponseDto`).

Error Responses:

Status	Condition
`400`	Refresh token is expired, revoked, or does not exist
`401`	Associated user account has been deactivated

2.6 Revoke Token

> `POST /api/auth/revoke-token` — `application/json`

Explicitly revokes a specific refresh token. Use this to sign out from a specific device while remaining logged in on others. The requester's IP address is automatically recorded.

Authorization: Required

Request Body:

```
{
  "refreshToken": "dGhpcyBpcyBhIHJlZnJlc2ggdG9rZW4..."
}
```

Success Response — `200 OK` **:** Confirmation message.

2.7 Forgot Password

`POST /api/auth/forget-password` — `application/json`

Initiates the password reset flow by sending a one-time password (OTP) to the provided email address. The OTP is time-limited and single-use.

Authorization: None (public endpoint)

Request Body:

```
{
  "email": "ahmed@example.com"
}
```

Note: This endpoint always returns `200 OK` regardless of whether the email exists, to prevent user enumeration attacks.

2.8 Verify Reset Password OTP

`POST /api/auth/verify-reset-otp` — `application/json`

Verifies the OTP received via email. On success, returns a token that must be included in the subsequent Reset Password request.

Authorization: None (public endpoint)

Request Body:

```
{
  "email": "ahmed@example.com",
  "otp": "483920"
}
```

2.9 Reset Password

> `POST /api/auth/reset-password` — `application/json`

Sets a new password using the token received after OTP verification. Once used, the token is invalidated.

Authorization: None (public endpoint)

Request Body:

```
{
  "email": "ahmed@example.com",
  "token": "CfDJ8KQ...",
  "newPassword": "NewSecure1!",
  "confirmPassword": "NewSecure1!"
}
```

2.10 Verify Phone Number

> `POST /api/auth/verify-phone` — `application/json`

Verifies the user's phone number using an OTP sent by SMS. Phone verification is required to complete the registration process.

Authorization: None

Request Body:

```
{
  "phoneNumber": "01012345678",
  "otp": "837261"
}
```

2.11 Resend Phone Code

> `POST /api/auth/resend-phone-code` — `application/json`

Resends the phone verification OTP. Subject to rate limiting.

Authorization: None

Request Body:

```
{
  "phoneNumber": "01012345678"
}
```

3. Account Management

3.1 Get My Account

> `GET /api/auth/me`

Returns the authenticated user's account details.

Authorization: Required

Success Response — `200 OK` :

```
{
  "success": true,
  "message": "OK",
  "data": {
    "userId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
    "firstName": "Ahmed",
    "lastName": "Hassan",
    "email": "ahmed@example.com",
    "phoneNumber": "01012345678",
    "phoneNumberConfirmed": true,
    "emailConfirmed": true,
    "role": "User",
    "profileImageUrl": "https://cdn.example.com/avatars/ahmed.jpg"
  }
}
```

3.2 Update My Account

> `PUT /api/auth/me` — `application/json`

Updates the authenticated user's name and phone number.

Authorization: Required

Request Body:

```
{
  "firstName": "Ahmed",
  "lastName": "Hassan",
  "phoneNumber": "01012345678"
}
```

Success Response — `200 OK` : Returns the updated `AccountDto`.

3.3 Change Password

> **`POST /api/auth/change-password`** — **`application/json`**

Changes the authenticated user's password. Requires the current password for verification.

Authorization: Required

Request Body:

```
{
  "currentPassword": "OldPass1!",
  "newPassword": "NewPass2@",
  "confirmPassword": "NewPass2@"
}
```

Field	Validation
`currentPassword`	Required; must match the stored password
`newPassword`	Required; minimum 8 characters
`confirmPassword`	Must exactly match `newPassword`

3.4 Request Email Update

> **`POST /api/auth/me/email`** — **`application/json`**

Initiates an email address change. A verification link is sent to the new address. The change is not applied until the link is confirmed.

Authorization: Required

Request Body:

```
{
  "newEmail": "new.email@example.com"
}
```

3.5 Cancel Email Update

> `POST /api/auth/me/email/cancel`

Cancels a pending email change request before it has been confirmed.

Authorization: Required

Request Body: None

3.6 Delete My Account

> `DELETE /api/auth/me`

Permanently deletes the authenticated user's account and all associated data. This action is irreversible.

Authorization: Required

> **Warning:** This operation is permanent and cannot be undone. All health profiles, documents, conversations, and appointments linked to this account will be removed.

4. Patient Profiles

A **Patient Profile** (also called a Health Profile) is the central data entity in Rafiq. It holds demographic information, allergies, chronic diseases, and links to all medical records. One user can own multiple profiles (e.g., a profile for themselves and one for a child). Profiles can also be shared with family members and caregivers.

Access Roles

Role	Permissions
Owner	Full read/write access; can invite/remove members; can delete the profile
Manager	Read/write access to all medical data; can manage members
Viewer	Read-only access to all profile data

4.1 Create Patient Profile

> `POST /api/patient-profiles` — `application/json`

Creates a new health profile for the currently authenticated user. Each user can only have one profile linked to their account. For additional profiles (e.g., for a child), use the **Create Managed Profile** endpoint.

Authorization: Required

Request Body:

```
{
  "dateOfBirth": "1990-05-15",
  "gender": 1,
  "bloodType": 2,
  "height": 175.5,
  "weight": 70.0,
  "allergies": [
    { "name": "Penicillin", "severity": "High" }
  ],
  "chronicDiseases": [
    { "name": "Type 2 Diabetes", "status": "Active", "diagnosedAt": "2018-03-01" }
  ]
}
```

Field	Type	Required	Description
`dateOfBirth`	`DateOnly`		Patient's date of birth in `YYYY-MM-DD` format
`gender`	`int`		`1` = Male, `2` = Female
`bloodType`	`int`		See Blood Type enum values below
`height`	`decimal`		Height in centimeters
`weight`	`decimal`		Weight in kilograms
`allergies`	`array`		Initial list of known allergies
`chronicDiseases`	`array`		Initial list of chronic conditions

Blood Type Values: `1` = A+, `2` = A−, `3` = B+, `4` = B−, `5` = AB+, `6` = AB−, `7` = O+, `8` = O−

Success Response — `201 Created`: Returns `PatientProfileDto`.

4.2 Create Managed Profile

> **`POST /api/patient-profiles/managed`** — **`application/json`**

Creates a health profile for a dependent (e.g., a child or elderly parent). Managed profiles are not linked to any user account directly — the creator becomes their **Owner**. The request body additionally requires the patient's name.

Authorization: Required

Request Body: Same as Create Patient Profile, plus:

```
{
  "firstName": "Sara",
  "lastName": "Hassan",
  "dateOfBirth": "2015-08-22"
}
```

Success Response — `201 Created`: Returns `PatientProfileDto`.

4.3 Get My Patient Profile

> **`GET /api/patient-profiles/me`**

Returns the health profile linked to the authenticated user's account.

Authorization: Required

Success Response — `200 OK`: Returns `PatientProfileDto`.

4.4 Get Accessible Health Profiles

> `GET /api/patient-profiles/accessible`

Returns all health profiles the authenticated user has access to, either as an owner, manager, or viewer. Use this endpoint to populate the profile-switcher in the app.

Authorization: Required

Success Response — `200 OK`: Returns a list of `PatientProfileDto`.

4.5 Get Profile by ID

> `GET /api/patient-profiles/{id}`

Returns a specific health profile by its ID. The caller must have at least **Viewer** access to the requested profile.

Authorization: Required

Path Parameters:

Parameter	Type	Description
`id`	`guid`	The health profile's unique identifier

Success Response — `200 OK`: Returns `PatientProfileDto`.

4.6 Update Patient Profile

> `PUT /api/patient-profiles/{id}` — `application/json`

Updates demographic information on an existing health profile. The caller must have **Manager** or **Owner** access. The request body must include the `patientProfileId` matching the route `{id}`.

Authorization: Required (Manager/Owner)

Request Body:

```
{
  "patientProfileId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
  "dateOfBirth": "1990-05-15",
  "gender": 1,
  "bloodType": 7,
  "height": 176.0,
  "weight": 72.5
}
```

Success Response — `200 OK` : Returns the updated `PatientProfileDto` .

4.7 Delete Patient Profile

> `DELETE /api/patient-profiles/{id}`

Permanently deletes a health profile and all associated medical data. Only the **Owner** can perform this action.

Authorization: Required (Owner only)

Path Parameters:

Parameter	Type	Description
`id`	`guid`	Health profile ID to delete

> **Caution:** All lab reports, imaging reports, prescriptions, medicines, appointments, and AI conversations linked to this profile will be permanently removed.

4.8 Update Profile Image

> `POST /api/patient-profiles/{id}/image` — `multipart/form-data`

Uploads or replaces the profile photo for a health profile. Passing `removeImage=true` without a file will delete the existing image.

Authorization: Required (Manager/Owner)

Request Body (Form Fields):

Field	Type	Required	Description
`profileImage`	`file`		Image file (JPEG, PNG, WEBP, GIF; max 5 MB)
`removeImage`	`boolean`		Set to `true` to remove the existing image

Success Response — `200 OK` **:** Returns the updated `PatientProfileDto`.

4.9 Send Health Profile Invitation

> `POST /api/patient-profiles/{profileId}/invitations` — `application/json`

Invites another registered Rafiq user to access a health profile. The invitation is sent to the specified email address. If the recipient accepts, they gain the specified role.

Authorization: Required (Owner/Manager)

Path Parameters:

Parameter	Type	Description
`profileId`	`guid`	The health profile to share

Request Body:

```
{
  "email": "family@example.com",
  "role": 3
}
```

Field	Type	Required	Description
`email`	`string`		Email address of the person to invite
`role`	`int`		`2` = Manager, `3` = Viewer

Success Response — `201 Created` **:** Invitation created.

> **Business Rule:** You cannot invite the profile owner themselves. An owner cannot be downgraded via invitation.

4.10 Get Received Invitations

> `GET /api/patient-profiles/invitations/received`

Returns all pending health profile invitations addressed to the authenticated user.

Authorization: Required

4.11 Get Sent Invitations

> `GET /api/patient-profiles/invitations/sent`

Returns all invitations the authenticated user has sent.

Authorization: Required

4.12 Accept Invitation

> `POST /api/patient-profiles/invitations/{invitationId}/accept`

Accepts a received invitation and grants the authenticated user access to the health profile with the assigned role.

Authorization: Required

4.13 Reject Invitation

> `POST /api/patient-profiles/invitations/{invitationId}/reject`

Declines a received health profile invitation.

Authorization: Required

4.14 Cancel Invitation

> `POST /api/patient-profiles/invitations/{invitationId}/cancel`

Cancels a sent invitation before it has been accepted. Only the sender can cancel.

Authorization: Required

4.15 Request Profile Access

> `POST /api/patient-profiles/access-requests` — `application/json`

Sends an unsolicited access request to the owner of a specific health profile. The owner must approve the request for access to be granted.

Authorization: Required

Request Body:

```
{
  "userHealthProfileId": "3fa85f64-5717-4562-b3fc-2c963f66afa6"
}
```

4.16 Get Received Access Requests

`GET /api/patient-profiles/access-requests/received`

Returns all pending access requests directed at health profiles owned by the authenticated user.

Authorization: Required

4.17 Accept Access Request

`POST /api/patient-profiles/access-requests/{requestId}/accept`

Approves a pending access request, granting the requester default **Viewer** access.

Authorization: Required (Profile Owner)

4.18 Reject Access Request

`POST /api/patient-profiles/access-requests/{requestId}/reject`

Rejects a pending access request.

Authorization: Required (Profile Owner)

4.19 Get Profile Members

`GET /api/patient-profiles/{profileId}/members`

Returns a list of all users who currently have access to the specified health profile, along with their roles and access status.

Authorization: Required (any member)

4.20 Change Member Role

`PATCH /api/patient-profiles/{profileId}/members/{accessId}/role` — `application/json`

Changes the access role of an existing member.

Authorization: Required (Owner only)

Request Body:

```
{
  "role": 2
}
```

Business Rule: The profile owner's role cannot be changed.

4.21 Revoke Member Access

`POST /api/patient-profiles/{profileId}/members/{accessId}/revoke`

Removes a member's access to the health profile.

Authorization: Required (Owner only)

4.22 Leave Health Profile

`POST /api/patient-profiles/{profileId}/leave`

Allows a non-owner member to voluntarily remove themselves from a health profile.

Authorization: Required (non-Owner members only)

4.23 Add Allergy

`POST /api/patient-profiles/{profileId}/allergies` — `application/json`

Adds a new allergy entry to the health profile. The `patientProfileId` in the body must match the route `{profileId}`.

Authorization: Required (Manager/Owner)

Request Body:

```
{
  "patientProfileId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
  "name": "Penicillin",
  "severity": "High"
}
```

Field	Type	Required	Description
`patientProfileId`	`guid`		Must match route `{profileId}`
`name`	`string`		Allergen name (e.g., "Penicillin", "Peanuts")
`severity`	`string`		`"Low"`, `"Medium"`, or `"High"`

Success Response — `201 Created`: Returns `AllergyDto`.

4.24 Update Allergy

> **`PUT /api/patient-profiles/{profileId}/allergies/{allergyId}`** — **`application/json`**

Updates an existing allergy record. Both the `patientProfileId` and `allergyId` in the request body must match the corresponding route parameters.

Authorization: Required (Manager/Owner)

4.25 Delete Allergy

> **`DELETE /api/patient-profiles/{profileId}/allergies/{allergyId}`**

Removes an allergy record from the health profile.

Authorization: Required (Manager/Owner)

4.26 Add Chronic Disease

> **`POST /api/patient-profiles/{profileId}/chronic-diseases`** — **`application/json`**

Adds a chronic condition to the health profile.

Authorization: Required (Manager/Owner)

Request Body:

```
{
  "patientProfileId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
  "name": "Type 2 Diabetes",
  "status": "Active",
  "diagnosedAt": "2018-03-01"
}
```

Field	Type	Required	Description
`name`	`string`		Disease or condition name
`status`	`string`		`"Active"`, `"Managed"`, or `"Resolved"`
`diagnosedAt`	`DateOnly`		Date of first diagnosis in `YYYY-MM-DD` format

Success Response — `201 Created`: Returns `ChronicDiseaseDto`.

4.27 Update Chronic Disease

> **`PUT /api/patient-profiles/{profileId}/chronic-diseases/{diseaseId}`** — **`application/json`**

Updates a chronic disease record.

Authorization: Required (Manager/Owner)

4.28 Delete Chronic Disease

> **`DELETE /api/patient-profiles/{profileId}/chronic-diseases/{diseaseId}`**

Removes a chronic disease record.

Authorization: Required (Manager/Owner)

5. Emergency Contacts

Emergency contacts are personal contact entries stored at the user-account level (not tied to a specific health profile). They are displayed to first responders or other users who need to reach someone in an emergency.

5.1 Add Emergency Contact

> `POST /api/emergency-contacts` — `application/json`

Authorization: Required

Request Body:

```
{
  "name": "Fatima Hassan",
  "phoneNumber": "01234567890",
  "relation": "Mother"
}
```

Field	Type	Required	Validation
`name`	`string`		Required; max 100 characters
`phoneNumber`	`string`		Valid Egyptian mobile number `^01[0125][0-9]{8}$`
`relation`	`string`		Required; max 100 characters (e.g., "Mother", "Spouse")

Success Response — `201 Created` : Returns `EmergencyContactResponseDto` .

5.2 Get Emergency Contacts

> `GET /api/emergency-contacts`

Returns all emergency contacts for the authenticated user.

Authorization: Required

Success Response — `200 OK` : Returns a list of `EmergencyContactResponseDto` .

```
{
  "success": true,
  "data": [
    {
      "id": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
      "userId": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
      "name": "Fatima Hassan",
      "phoneNumber": "01234567890",
      "relation": "Mother"
    }
  ]
}
```


5.3 Delete Emergency Contact

`DELETE /api/emergency-contacts/{id}`

Removes an emergency contact.

Authorization: Required

Path Parameters:

Parameter	Type	Description
`id`	`guid`	Emergency contact ID

6. Medical Records — Lab Reports

Lab reports store the results of clinical laboratory tests. Each report can contain multiple individual test results. The AI engine automatically extracts data from uploaded report images using OCR and intelligent parsing.

Upload Flow

The typical workflow is: 1. **Upload** the image → AI extracts structured data → returns raw extracted fields. 2. **Save** the structured data (after user review/correction) → persists the report. 3. **View** or **Update** the saved report as needed.

6.1 Upload Lab Report (AI Extraction)

> `POST /api/documents/upload/lab?profileId={profileId}` — `multipart/form-data`

Uploads a lab report image. The AI engine performs OCR and extracts structured data (lab name, doctor name, test results, date). The data is returned for user review but **not yet saved** to the database.

Authorization: Required

Query Parameters:

Parameter	Type	Required	Description
`profileId`	`guid`		The health profile to associate this report with

Request Body: Form field `image` — image file (JPEG, PNG)

Success Response — `200 OK` : Returns the AI-extracted data in `LabReportResponseDto` format for client-side review.

6.2 Save Lab Report

> `POST /api/documents/labs?profileId={profileId}` — `application/json`

Persists a lab report (typically after the user has reviewed and corrected the AI-extracted data from the upload step).

Authorization: Required

Query Parameters:

Parameter	Type	Required	Description
`profileId`	`guid`		The health profile to associate this report with

Request Body:

```
{
  "labName": "Cairo Scan Center",
  "doctorName": "Dr. Mostafa Kamel",
  "reportDate": "2026-07-15",
  "summary": "All values within normal range.",
  "ocrText": "...",
  "imageUrl": "https://cdn.example.com/labs/report.jpg",
  "results": [
    {
      "testName": "Hemoglobin",
      "value": "13.5",
      "unit": "g/dL",
      "normalRange": "12.0 – 16.0",
      "status": "Normal"
    }
  ]
}
```

Success Response — `201 Created` : Returns the saved `LabReportResponseDto` .

6.3 Get My Lab Reports

> `GET /api/documents/labs?profileId={profileId}`

Returns all lab reports for the specified health profile.

Authorization: Required (any member with profile access)

Success Response — `200 OK` : Returns a list of `LabReportResponseDto` .

6.4 Get Lab Report by ID

> `GET /api/documents/labs/{id}`

Returns a single lab report with all its test results.

Authorization: Required

6.5 Update Lab Report

> `PUT /api/documents/labs/{id}` — `application/json`

Updates an existing lab report. All fields are replaced with the submitted values.

Authorization: Required (Manager/Owner)

6.6 Delete Lab Report

> `DELETE /api/documents/labs/{id}`

Permanently deletes a lab report and all its associated test results.

Authorization: Required (Manager/Owner)

7. Medical Records — Imaging Reports

Imaging reports cover radiology results such as X-rays, MRIs, CT scans, and ultrasounds. Like lab reports, they support AI-assisted extraction from uploaded images.

7.1 Upload Imaging Report (AI Extraction)

> `POST /api/documents/upload/imaging?profileId={profileId}` — `multipart/form-data`

Uploads an imaging report. The AI engine extracts imaging type, body part, findings, impression, and doctor information. Returns extracted data for review — does **not** save automatically.

Authorization: Required

7.2 Save Imaging Report

> `POST /api/documents/imaging?profileId={profileId}` — `application/json`

Persists the imaging report after user review.

Authorization: Required

Request Body:

```
{
  "imagingType": "MRI",
  "bodyPart": "Brain",
  "findings": "No abnormal signal intensities detected.",
  "impression": "Normal brain MRI.",
  "doctorName": "Dr. Sara Nour",
  "reportDate": "2026-07-20",
  "summary": "Brain MRI shows no abnormalities.",
  "ocrText": "...",
  "imageUrl": "https://cdn.example.com/imaging/brain-mri.jpg"
}
```

Success Response — `201 Created`: Returns `ImagingReportResponseDto`.

7.3 Get My Imaging Reports

> `GET /api/documents/imaging?profileId={profileId}`

Returns all imaging reports for a health profile.

Authorization: Required

7.4 Get Imaging Report by ID

> `GET /api/documents/imaging/{id}`

Returns a single imaging report.

Authorization: Required

7.5 Update Imaging Report

> `PUT /api/documents/imaging/{id}` — `application/json`

Updates an existing imaging report.

Authorization: Required (Manager/Owner)

7.6 Delete Imaging Report

> `DELETE /api/documents/imaging/{id}`

Permanently deletes an imaging report.

Authorization: Required (Manager/Owner)

8. Medical Records — General Documents

General documents serve as a catch-all category for any medical paperwork that does not fit into the structured Lab or Imaging categories — discharge summaries, consultation notes, referral letters, vaccination records, and more. The AI engine classifies, summarizes, and extracts key information from uploaded documents.

Two upload modes are available: **synchronous** (blocks until AI completes) and **asynchronous** (returns immediately; AI runs in the background).

8.1 Upload General Document (Synchronous)

> `POST /api/documents/general/upload` — `multipart/form-data`

Uploads a document and waits for AI analysis to complete before responding. Suitable for smaller documents where latency is acceptable.

Authorization: Required

Request Body (Form Fields):

Field	Type	Required	Description
`profileId`	`guid`		Health profile to associate with
`image`	`file`		Document image

8.2 Upload General Document (Asynchronous)

> `POST /api/documents/general/upload-async?profileId={profileId}` — `multipart/form-data`

Uploads a document and immediately returns a `documentId`. AI analysis runs as a background job. The client should either: - Poll `GET /api/documents/general/status/{id}` to check progress, or - Listen to the `DocumentAnalysisCompleted` SignalR event.

Authorization: Required

Query Parameters:

Parameter	Type	Required	Description
`profileId`	`guid`		The health profile to associate with

Form Fields:

Field	Type	Required	Description
`image`	`file`		Document image
`description`	`string`		Optional user description hint for the AI

Success Response — `200 OK` :

```
{
  "success": true,
  "data": {
    "documentId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
    "imagePath": "https://cdn.example.com/docs/doc123.jpg",
    "title": "Uploaded Document"
  }
}
```

8.3 Get Document Analysis Status

> `GET /api/documents/general/status/{id}`

Returns the current AI analysis status of a specific general document. Use as a polling fallback when SignalR is unavailable.

Authorization: Required

Path Parameters:

Parameter	Type	Description
`id`	`guid`	General document ID

Success Response — `200 OK` :

```
{
  "success": true,
  "data": {
    "id": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
    "analysisStatus": "Processing",
    "title": "Consultation Note",
    "aiSummary": null
  }
}
```

Analysis Status Values:

Status	Meaning
`Pending`	Queued, AI has not started yet
`Processing`	AI is currently analyzing
`Completed`	Analysis finished successfully
`Failed`	AI analysis encountered an error

8.4 Retry Document Analysis

> **`POST /api/documents/general/{id}/retry`**

Re-queues a document that is in the `Failed` state for another AI analysis attempt.

Authorization: Required

> **Note:** Returns **`400 Bad Request`** if the document is not in **`Failed`** state.

8.5 Save General Document

> **`POST /api/documents/general?profileId={profileId}`** — **`application/json`**

Manually saves a general document record (without AI extraction). Use this when the user wants to store a document with hand-entered metadata.

Authorization: Required

Request Body:

```
{
  "title": "Hospital Discharge Summary",
  "description": "Post-operation discharge summary from Cairo Hospital.",
  "aiSummary": null,
  "imagePath": "https://cdn.example.com/docs/discharge.jpg",
  "documentType": "DischargeSummary",
  "doctorName": "Dr. Khaled Sayed",
  "hospitalOrClinic": "Cairo University Hospital",
  "documentDate": "2026-06-10",
  "ocrText": null
}
```

Success Response — `201 Created` : Returns `GeneralDocumentResponseDto` .

8.6 Get My General Documents

> `GET /api/documents/general?profileId={profileId}`

Returns all general documents for the specified health profile.

Authorization: Required

8.7 Update General Document

> `PUT /api/documents/general/{id}` — `application/json`

Updates an existing general document's metadata.

Authorization: Required (Manager/Owner)

8.8 Delete General Document

> `DELETE /api/documents/general/{id}`

Permanently deletes a general document.

Authorization: Required (Manager/Owner)

8.9 Upload Record Image (No AI)

> `POST /api/documents/upload-image` — `multipart/form-data`

Uploads an image file to storage without triggering any AI processing. Returns the stored image URL for use when manually creating records. The image is **not** saved to the database — the returned path must be included in a subsequent save request.

Authorization: Required

Request Body: Form field `image` — image file

Success Response — `200 OK` :

```
{
  "success": true,
  "data": { "imagePath": "https://cdn.example.com/manual/img123.jpg" }
}
```

9. Medical Report PDF

9.1 Generate Medical Report PDF

```
GET /api/medical-report/{profileId}?reportType={type}
```

Generates a PDF summary of the health profile's complete medical history. The report is returned as a binary PDF file stream for download.

Authorization: Required

Path Parameters:

Parameter	Type	Description
`profileId`	`guid`	The health profile to report on

Query Parameters:

Parameter	Type	Default	Description
`reportType`	`int`	`1`	`1` = Doctor Summary (concise), `2` = Complete History (full detail)

Success Response — `200 OK`:

```
Content-Type: application/pdf
Content-Disposition: attachment; filename="RafiqMedicalReport_DoctorSummary_2026-08-04.pdf"
```

> **Tip:** Use the Doctor Summary type when sharing with a physician; use Complete History for a comprehensive personal archive.

10. Prescriptions

Prescriptions store digitized copies of doctor-issued medication prescriptions. Each prescription contains a list of medicines with their dosage instructions. Prescriptions support AI-powered extraction from uploaded images and can be used to auto-populate the user's medicine list.

10.1 Upload Prescription (AI Extraction)

> `POST /api/prescriptions/upload?profileId={profileId}` — `multipart/form-data`

Uploads a prescription image. The AI extracts doctor name, patient name, prescription date, and the full list of medicines with dosages, frequencies, and durations. Returns extracted data for review.

Authorization: Required

10.2 Save Prescription

> `POST /api/prescriptions?profileId={profileId}` — `application/json`

Persists a prescription (after AI extraction review or manual entry).

Authorization: Required

Request Body:

```
{
  "doctorName": "Dr. Omar Farouk",
  "patientName": "Ahmed Hassan",
  "prescriptionDate": "2026-07-28",
  "imagePath": "https://cdn.example.com/prescriptions/rx123.jpg",
  "medicines": [
    {
      "medicineName": "Augmentin",
      "dosage": "875mg",
      "frequency": "Twice daily",
      "duration": "7 days",
      "instructions": "Take with food",
      "notes": null
    }
  ]
}
```

Success Response — `201 Created` : Returns `PrescriptionResponseDto` .

10.3 Get My Prescriptions

`GET /api/prescriptions?profileId={profileId}`

Returns all prescriptions for the specified health profile, ordered by date descending.

Authorization: Required

10.4 Get Prescription by ID

`GET /api/prescriptions/{id}`

Returns a single prescription with all its medicine items.

Authorization: Required

10.5 Update Prescription

`PUT /api/prescriptions/{id}` — `application/json`

Updates the header fields of a prescription (doctor name, patient name, date). Medicine items must be managed separately.

Authorization: Required (Manager/Owner)

Request Body:

```
{
  "doctorName": "Dr. Omar Farouk",
  "patientName": "Ahmed Hassan",
  "prescriptionDate": "2026-07-28"
}
```

10.6 Delete Prescription

`DELETE /api/prescriptions/{id}`

Permanently deletes a prescription and all its medicine items.

Authorization: Required (Manager/Owner)

11. User Medicines

User Medicines represent the patient's active medication list. Medicines can be added manually, imported from a saved prescription, or extracted from a scanned medicine box using AI. Allergy safety checking is built in.

11.1 Check Medication Allergy Safety

> `POST /api/user-medicines/check-allergy` — `application/json`

Before adding a new medication, call this endpoint to check whether it may cause an allergic reaction based on the patient's recorded allergies. Powered by AI.

Authorization: Required

Request Body:

```
{
  "profileId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
  "medicationName": "Amoxicillin"
}
```

Success Response — `200 OK` :

```
{
  "success": true,
  "data": {
    "isSafe": false,
    "riskLevel": "High",
    "reason": "Patient has a documented Penicillin allergy. Amoxicillin is a penicillin-type antibiotic."
  }
}
```

> **Note:** If no allergies are on file for the profile, the endpoint always returns `isSafe: true` with `riskLevel: "None"`. On AI failure, a safe result is returned so the user is never blocked from adding a medicine.

11.2 Upload Medicine Box Image (No AI)

> `POST /api/user-medicines/upload-image` — `multipart/form-data`

Uploads a medicine box photo to storage without AI processing. Returns the image URL for use in a subsequent manual Add Medicine request.

Authorization: Required

Success Response — `200 OK` :

```
{
  "success": true,
  "data": { "imagePath": "https://cdn.example.com/medicine-boxes/box123.jpg" }
}
```

11.3 Scan Medicine Box (AI Extraction)

> `POST /api/user-medicines/scan-box` — `multipart/form-data`

Uploads a medicine box image and uses AI vision to extract medicine name, dosage, frequency, duration, and other details. Returns extracted data for user review — does **not** save to the database.

Authorization: Required

Request Body: Form field `image` — medicine box photo

Success Response — `200 OK` : Returns AI-extracted medicine details for review.

11.4 Add Medicine from Prescription

> `POST /api/user-medicines/from-prescription?profileId={profileId}` — `application/json`

Bulk-imports selected medicines from a saved prescription into the user's medicine list.

Authorization: Required

Query Parameters:

Parameter	Type	Required	Description
`profileId`	`guid`		Target health profile

Request Body:

```
{
  "prescriptionMedicineIds": [
    "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
    "b2c3d4e5-f6a7-8901-bcde-f01234567891"
  ]
}
```

Success Response — `201 Created`: Returns a list of created `UserMedicineResponseDto`.

11.5 Add Medicine Manually

`POST /api/user-medicines?profileId={profileId}` — **`application/json`**

Adds a single medicine to the patient's medication list.

Authorization: Required

Request Body:

```
{
  "medicineName": "Metformin",
  "dosage": "500mg",
  "frequency": "Twice daily",
  "duration": "Ongoing",
  "notes": "Take with meals to reduce stomach upset.",
  "imagePath": null,
  "source": 1
}
```

Field	Type	Required	Description
`medicineName`	`string`	▫	Medicine name
`dosage`	`string`	▫	Dosage amount (e.g., "500mg")
`frequency`	`string`	▫	Administration frequency (e.g., "Twice daily")
`duration`	`string`	▫	Treatment duration (e.g., "7 days", "Ongoing")
`notes`	`string`		Optional notes
`imagePath`	`string`		URL from a previous image upload
`source`	`int`		`1` = Manual, `2` = Prescription, `3` = MedicineBox

Success Response — `201 Created`: Returns `UserMedicineResponseDto`.

11.6 Get My Medicines

`GET /api/user-medicines?profileId={profileId}`

Returns all medicines in the patient's medication list.

Authorization: Required

11.7 Get Medicine by ID

`GET /api/user-medicines/{id}`

Returns a single medicine record.

Authorization: Required

11.8 Update Medicine

`PUT /api/user-medicines/{id}` — `application/json`

Updates an existing medicine record's details.

Authorization: Required (Manager/Owner)

Request Body:

```
{
  "medicineName": "Metformin",
  "dosage": "1000mg",
  "frequency": "Once daily",
  "duration": "Ongoing",
  "notes": "Dosage increased by Dr. Kamel on 2026-08-01."
}
```

11.9 Delete Medicine

`DELETE /api/user-medicines/{id}`

Soft-deletes a medicine from the medication list.

Authorization: Required (Manager/Owner)

12. Medication Reminder Schedules

Medication reminder schedules define **when** and **how often** a reminder should fire for a specific medicine. They are the configuration layer — the Medication Reminder Engine (Section 13) handles the actual reminder logs and user interactions.

12.1 Create Reminder Schedules

> `POST /api/user-medicines/{medicineId}/reminders` — `application/json`

Creates one or more reminder schedule entries for a medicine. Each entry in the `times` array creates a separate reminder at that clock time each day.

Authorization: Required

Path Parameters:

Parameter	Type	Description
`medicineId`	`guid`	The medicine to attach reminders to

Request Body:

```
{
  "userMedicineId": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
  "startDate": "2026-08-05",
  "endDate": "2026-08-12",
  "repeatType": "Daily",
  "times": ["08:00:00", "20:00:00"]
}
```

Field	Type	Required	Description
`userMedicineId`	`guid`		Must match route `{medicineId}`
`startDate`	`DateOnly`		First day reminders should fire
`endDate`	`DateOnly`		Last day reminders should fire; must be ≥ `startDate`
`repeatType`	`string`		`"Once"`, `"Daily"`, `"Weekly"`, or `"Monthly"`
`times`	`string[]`		Array of `HH:mm:ss` clock times for daily reminders

Success Response — `200 OK`: Returns a list of created `MedicineReminderResponseDto`.

12.2 Get Reminder Schedules for a Medicine

> `GET /api/user-medicines/{medicineId}/reminders`

Returns all reminder schedules configured for a specific medicine.

Authorization: Required

12.3 Get Reminder Schedule by ID

> `GET /api/medicine-reminders/{id}`

Returns a single reminder schedule by its ID.

Authorization: Required

12.4 Update Reminder Schedule

> `PUT /api/medicine-reminders/{id}` — `application/json`

Updates an existing reminder schedule. The `id` in the body must match the route parameter.

Authorization: Required

12.5 Delete Reminder Schedule

> `DELETE /api/medicine-reminders/{id}`

Permanently deletes a reminder schedule.

Authorization: Required

12.6 Toggle Reminder Status

> `PATCH /api/medicine-reminders/{id}/toggle-status`

Enables or disables a reminder schedule. Disabled reminders will not trigger notifications.

Authorization: Required

Success Response — `200 OK` : Returns `true` if the reminder is now enabled, `false` if now disabled.

13. Medication Reminder Engine

The Medication Reminder Engine manages the runtime logs generated when a scheduled reminder fires. Each day, the engine creates reminder log entries based on active schedules. Users can confirm, skip, or snooze each reminder.

13.1 Get Today's Medication Reminders

> `GET /api/medication-reminders/today?profileId={profileId}`

Returns all reminder logs scheduled for today for the specified health profile. Each entry includes an `isActionable` flag — exactly one log per medicine per day is actionable at any given time.

Authorization: Required

Query Parameters:

Parameter	Type	Required	Description
`profileId`	`guid`		Health profile to query reminders for

Success Response — `200 OK` : Returns a list of `MedicationReminderLogDto` .

```
{
  "success": true,
  "data": [
    {
      "id": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
      "medicineReminderId": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
      "userHealthProfileId": "b2c3d4e5-f6a7-8901-bcde-f01234567891",
      "medicineName": "Metformin",
      "dosage": "500mg",
      "reminderTime": "08:00:00",
      "scheduledDate": "2026-08-04",
      "scheduledTime": "08:00:00",
      "reminderNumber": 1,
      "status": "Pending",
      "isActionable": true,
      "sentAt": "2026-08-04T08:00:00Z",
      "confirmedAt": null,
      "createdAt": "2026-08-04T05:00:00Z"
    }
  ]
}
```

13.2 Get Medication Reminder History

> `GET /api/medication-reminders/history?profileId={profileId}&date={date}`

Returns the reminder history for a specific date.

Authorization: Required

Query Parameters:

Parameter	Type	Required	Description
`profileId`	`guid`		Health profile
`date`	`DateOnly`		Date to retrieve history for (`YYYY-MM-DD`)

13.3 Get Reminder by ID

> `GET /api/medication-reminders/{id}`

Returns a single reminder log entry by its ID.

Authorization: Required

13.4 Confirm Medication Intake

> `POST /api/medication-reminders/{id}/confirm`

Marks a medication dose as taken. Only actionable reminders can be confirmed.

Authorization: Required

Path Parameters:

Parameter	Type	Description
`id`	`guid`	Reminder log ID (`ReminderLogId`)

Success Response — `200 OK` : Confirmation message.

13.5 Skip Medication Reminder

> `POST /api/medication-reminders/{id}/skip`

Marks a medication dose as intentionally skipped.

Authorization: Required

13.6 Snooze Medication Reminder

> `POST /api/medication-reminders/{id}/snooze` — `application/json`

Postpones a medication reminder by the specified number of minutes. The reminder will re-appear after the snooze period.

Authorization: Required

Request Body:

```
{
  "snoozeMinutes": 30
}
```

Field	Type	Required	Description
`snoozeMinutes`	`int`		Number of minutes to snooze (must be positive)

14. Appointments

Appointments track scheduled healthcare visits, tests, and follow-ups. Each appointment supports a customizable reminder offset so the user is notified ahead of time.

14.1 Create Appointment

> `POST /api/appointments?profileId={profileId}` — `application/json`

Creates a new appointment.

Authorization: Required

Query Parameters:

Parameter	Type	Required	Description
`profileId`	`guid`		Health profile for this appointment

Request Body:

```
{
  "appointmentType": 1,
  "customType": null,
  "title": "Annual Checkup",
  "provider": "Dr. Hassan Clinic",
  "appointmentDateTime": "2026-09-15T10:30:00Z",
  "reminderOffsetMinutes": 60,
  "notes": "Bring previous lab results."
}
```

Field	Type	Required	Description
`appointmentType`	`int`	✓	See Appointment Type values below
`customType`	`string`	Conditional	Required only when `appointmentType` = `8` (Other)
`title`	`string`	✓	Short descriptive title
`provider`	`string`		Doctor or clinic name
`appointmentDateTime`	`datetime`	✓	Must be a future UTC datetime
`reminderOffsetMinutes`	`int`		Minutes before the appointment to send a notification; must be ≥ 0
`notes`	`string`		Optional free-text notes

Appointment Type Values:

Value	Meaning
`1`	Doctor Visit
`2`	Lab Test
`3`	Imaging
`4`	Vaccination
`5`	Dentist
`6`	Therapy
`7`	Follow-Up
`8`	Other (requires `customType`)

Success Response — `201 Created`: Returns `AppointmentResponseDto`.

14.2 Get All Appointments

> `GET /api/appointments?profileId={profileId}`

Returns all appointments for the specified health profile.

Authorization: Required

14.3 Get Upcoming Appointments

```
GET /api/appointments/upcoming?profileId={profileId}
```

Returns only future, non-cancelled appointments ordered chronologically.

Authorization: Required

14.4 Get Today's Appointments

```
GET /api/appointments/today?profileId={profileId}
```

Returns appointments scheduled for the current calendar day.

Authorization: Required

14.5 Get Appointment by ID

```
GET /api/appointments/{id}
```

Returns a single appointment by ID.

Authorization: Required

14.6 Update Appointment

`PUT /api/appointments/{id}` — `application/json`

Updates all fields of an existing appointment.

Authorization: Required (Manager/Owner)

14.7 Delete Appointment

```
DELETE /api/appointments/{id}
```

Permanently deletes an appointment.

Authorization: Required (Manager/Owner)

14.8 Complete Appointment

```
PATCH /api/appointments/{id}/complete
```

Marks an appointment as completed. A completed appointment can no longer be modified.

Authorization: Required (Manager/Owner)

14.9 Cancel Appointment

> `PATCH /api/appointments/{id}/cancel`

Cancels a scheduled appointment.

Authorization: Required (Manager/Owner)

15. AI Chat

The AI Chat feature provides a conversational health assistant. Users can ask health-related questions, and the AI responds with context-aware answers drawn from the patient's profile data (allergies, chronic diseases, medications, and history). The assistant operates asynchronously — the HTTP request returns immediately while the AI processes in the background; the response is delivered via SignalR.

15.1 Create AI Conversation

> `POST /api/chat/conversations` — `application/json`

Creates a new chat conversation session tied to a health profile.

Authorization: Required

Request Body:

```
{
  "userHealthProfileId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
  "title": "Medication Questions"
}
```

Success Response — `200 OK` : Returns `ConversationSummaryDto` .

15.2 Get My Conversations

> `GET /api/chat/conversations`

Returns all active (non-archived) conversations for the authenticated user, ordered by most recent activity. Used to populate the conversation sidebar. Does **not** include message content.

Authorization: Required

Success Response — `200 OK` : Returns a list of `ConversationSummaryDto` .

15.3 Get Conversation History

> `GET /api/chat/conversations/{conversationId}`

Returns the full message history for a conversation, including all user and assistant messages with their statuses.

Authorization: Required

Success Response — `200 OK` : Returns `ConversationHistoryDto` .

```
{
  "success": true,
  "data": {
    "id": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
    "title": "Medication Questions",
    "lastMessageAt": "2026-08-04T18:00:00Z",
    "messages": [
      {
        "id": "msg-id-1",
        "role": "user",
        "content": "What does my current lab result mean?",
        "sequenceNumber": 1,
        "createdAt": "2026-08-04T17:58:00Z",
        "userReaction": null,
        "status": null
      },
      {
        "id": "msg-id-2",
        "role": "assistant",
        "content": "Based on your recent hemoglobin result of 13.5 g/dL...",
        "sequenceNumber": 2,
        "createdAt": "2026-08-04T18:00:00Z",
        "userReaction": "ThumbsUp",
        "status": "Completed"
      }
    ]
  }
}
```


15.4 Rename Conversation

`PATCH /api/chat/conversations/{conversationId}` — `application/json`

Renames a conversation.

Authorization: Required

Request Body:


```
{
  "title": "Diabetes Management"
}
```

15.5 Generate Conversation Title

> `POST /api/chat/conversations/{conversationId}/generate-title`

Uses AI to automatically generate a contextual title from the first exchange in the conversation.

Authorization: Required

15.6 Archive Conversation

> `DELETE /api/chat/conversations/{conversationId}`

Soft-deletes (archives) a conversation. Archived conversations no longer appear in the conversation list and cannot receive new messages.

Authorization: Required

15.7 Send Message

> `POST /api/chat/conversations/{conversationId}/messages` — `application/json`

Sends a message to the AI assistant. Returns `202 Accepted` immediately with placeholder IDs for the user and assistant messages. The actual AI response is delivered asynchronously via the `ChatResponse` SignalR event.

Authorization: Required

Request Body:

```
{
  "text": "Are my current medications safe with my penicillin allergy?",
  "base64Image": null,
  "imageFormat": null,
  "language": "en",
  "utcOffsetMinutes": 180
}
```

Field	Type	Required	Description
`text`	`string`		The user's message
`base64Image`	`string`		Base64-encoded image for multimodal queries
`imageFormat`	`string`		Image MIME type (e.g., `"image/jpeg"`) when `base64Image` is provided
`language`	`string`		Response language; `"en"` (default) or `"ar"`
`utcOffsetMinutes`	`int`		Client's UTC offset for time-aware responses

Success Response — `202 Accepted`:

```
{
  "userMessageId": "msg-user-id",
  "assistantMessageId": "msg-assistant-id"
}
```

> **SignalR Event:** When the AI finishes processing, the `ChatResponse` event is emitted on the user's connection with the full assistant message content.

15.8 React to Message

> `POST /api/chat/conversations/{conversationId}/messages/{messageId}/react` — `application/json`

Adds or removes a thumbs-up ( ) or thumbs-down ( ) reaction on an assistant message. Setting `remove: true` removes the current reaction.

Authorization: Required

Request Body:

```
{
  "reactionType": "ThumbsDown",
  "remove": false,
  "feedback": "The information provided was inaccurate."
}
```

15.9 Generate AI Health Summary

> `GET /api/chat/health-summary/{profileId}?language={lang}`

Generates an AI-powered health overview for the profile using all available data. Returns a structured summary with overall status, conditions, medication brief, lab result status, insights, and recommendations. Returns `hasData: false` when there is insufficient data to generate a meaningful summary.

Authorization: Required

Query Parameters:

Parameter	Type	Default	Description
`language`	`string`	`"en"`	`"en"` for English, `"ar"` for Arabic

Success Response — `200 OK` : Returns `HealthSummaryDto`.

15.10 Get AI Access Status

> `GET /api/chat/ai-status`

Returns whether the authenticated user's AI access is currently restricted. Called by the frontend before opening the chat UI to determine whether to lock the interface.

Authorization: Required

Success Response — `200 OK` :

```
{
  "success": true,
  "data": { "isAiRestricted": false }
}
```

16. Voice Agent

The Voice Agent provides a hands-free conversational interface powered by natural language processing. Like the AI Chat, it is health-context aware and operates on a session model.

16.1 Create Voice Session

> **`POST /api/voice-agent/sessions`** — **`application/json`**

Initializes a new voice agent session tied to a health profile. Returns a `sessionId` for subsequent message calls.

Authorization: Required

Request Body:

```
{
  "profileId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
  "language": "ar"
}
```

Field	Type	Required	Description
`profileId`	`guid`		Health profile providing context
`language`	`string`		Session language (`"en"` or `"ar"`); defaults to `"en"`

16.2 Send Voice Message (Synchronous)

> **`POST /api/voice-agent/sessions/{sessionId}/messages`** — **`application/json`**

Sends a text transcription to the voice agent and waits synchronously for the full AI response. Also emits SignalR events if the client is connected. Suitable for Postman, automated tests, and mobile integrations.

Authorization: Required

Request Body:

```
{
  "text": "ما هي أدويتي لليوم؟",
  "language": "ar",
  "utcOffsetMinutes": 180
}
```

16.3 Stream Voice Message (Asynchronous)

> **`POST /api/voice-agent/sessions/{sessionId}/messages/stream`** — **`application/json`**

Sends a text transcription and returns `202 Accepted` immediately. The agent loop runs in the background. The Angular voice interface receives responses exclusively through SignalR events:

SignalR Event	Payload	Description
`VoiceAgentThinking`	`{ toolName }`	Emitted before each tool call
`VoiceAgentResponse`	`{ sessionId, text, navigateTo, needsMoreInfo }`	Final answer
`VoiceAgentError`	`{ message }`	Unrecoverable failure

Authorization: Required

17. Speech Services

Speech services provide infrastructure for voice input/output without exposing third-party API keys to the client.

17.1 Get Azure Speech Token

> `POST /api/speech/token`

Issues a short-lived Azure Cognitive Services Speech token (valid for 10 minutes). The client uses this token to make direct calls to Azure Speech-to-Text, keeping the subscription key server-side.

Authorization: None (public endpoint)

Success Response — `200 OK` :

```
{
  "token": "eyJhbGciOiJSUz...",
  "region": "eastus",
  "expiresInSeconds": 600
}
```

17.2 Text-to-Speech via ElevenLabs

> `POST /api/speech/elevenlabs-tts` — `application/json`

Proxies text-to-speech conversion through ElevenLabs, returning an MP3 audio stream. Text must be under 2,000 characters. Returns `404 Not Found` if ElevenLabs is not configured — the frontend should fall back to Azure TTS in this case.

Authorization: None (public endpoint)

Request Body:

```
{
  "text": "Your appointment with Dr. Hassan is tomorrow at 10:30 AM."
}
```

Success Response — `200 OK` :

```
Content-Type: audio/mpeg
[Binary MP3 stream]
```

18. Notifications

Notifications are system-generated alerts for events such as incoming invitations, medication reminders, and appointment reminders. Real-time delivery is handled via SignalR; this API handles retrieval and read-state management.

18.1 Get My Notifications

> `GET /api/notifications?page={page}&pageSize={pageSize}`

Returns the authenticated user's notifications in reverse-chronological order.

Authorization: Required

Query Parameters:

Parameter	Type	Default	Description
`page`	`int`	`1`	Page number (1-indexed)
`pageSize`	`int`	`30`	Items per page

Success Response — `200 OK` :

```
{
  "success": true,
  "data": {
    "items": [
      {
        "id": "notif-id-1",
        "title": "Medication Reminder",
        "body": "Time to take your Metformin 500mg.",
        "titleAr": "تذكير دواء",
        "bodyAr": "حان وقت تناول دواء ميتفورمين 500 ملجم.",
        "type": "MedicationReminder",
        "isRead": false,
        "readAt": null,
        "createdAt": "2026-08-04T08:00:00Z"
      }
    ],
    "unreadCount": 3,
    "page": 1,
    "pageSize": 30
  }
}
```


18.2 Mark Notification as Read

`PATCH /api/notifications/{id}/read`

Marks a single notification as read.

Authorization: Required

18.3 Mark All Notifications as Read

`PATCH /api/notifications/read-all`

Marks all of the authenticated user's notifications as read.

Authorization: Required

19. Reviews

Users can submit reviews of the Rafiq application. Reviews include a star rating and an optional comment. Administrators can manage, respond to, and moderate reviews.

19.1 Submit Review

> `POST /api/reviews` — `application/json`

Submits a user review of the application.

Authorization: Required

Request Body:

```
{
  "stars": 5,
  "comment": "Excellent app! The AI health summary is incredibly helpful."
}
```

Field	Type	Required	Validation
`stars`	`int`		1–5
`comment`	`string`		Optional free-text feedback

19.2 Get Public Reviews

> `GET /api/reviews/public?limit={limit}`

Returns visible, approved reviews for display on the application's landing page.

Authorization: None (public endpoint)

Query Parameters:

Parameter	Type	Default	Description
`limit`	`int`	`20`	Maximum number of reviews to return

20. Administration

All administration endpoints are restricted to users with the **Admin** role. They provide oversight of users, AI operations, usage patterns, and review management.

20.1 Get Admin Dashboard

> `GET /api/v1/admin/dashboard`

Returns a comprehensive dashboard snapshot including user statistics, AI request counts, growth trends, gender distribution, and users flagged for attention.

Authorization: Required (Admin role)

Success Response — `200 OK` : Returns `AdminDashboardDto`.

20.2 List Users

> `GET /api/v1/admin/users`

Returns a paginated, filterable, and sortable list of all registered users.

Authorization: Required (Admin role)

Query Parameters:

Parameter	Type	Default	Description
`search`	`string`	—	Filter by name or email
`status`	`string`	—	`"active"` or `"inactive"`
`role`	`string`	—	`"User"` or `"Admin"`
`sortBy`	`string`	`"createdAt"`	Field to sort by
`sortDirection`	`string`	`"desc"`	`"asc"` or `"desc"`
`page`	`int`	`1`	Page number
`pageSize`	`int`	`20`	Items per page

Success Response — `200 OK` : Returns `PagedResult<AdminUserListItemDto>`.

20.3 Set User Status

> `PATCH /api/v1/admin/users/{userId}/status` — `application/json`

Activates or deactivates a user account. Deactivated users cannot log in.

Authorization: Required (Admin role)

Request Body:

```
{
  "isActive": false
}
```

20.4 Get Audit Logs

> `GET /api/v1/admin/audit-logs`

Returns a paginated log of all administrative and system actions for audit and compliance purposes.

Authorization: Required (Admin role)

Query Parameters:

Parameter	Type	Description
`search`	`string`	Free text search
`module`	`string`	Filter by system module
`severity`	`string`	Filter by severity level
`dateFrom`	`string`	Start date filter (`YYYY-MM-DD`)
`dateTo`	`string`	End date filter (`YYYY-MM-DD`)
`page`	`int`	Page number
`pageSize`	`int`	Items per page

20.5 Get AI Overview

> `GET /api/v1/admin/ai/overview`

Returns AI health metrics including a composite health score (0–100), positive reaction rate, unreviewed negative feedback count, failure rate, and a 7-day quality trend chart.

Authorization: Required (Admin role)

20.6 List AI Requests

> `GET /api/v1/admin/ai/requests`

Returns a paginated log of all AI requests with filtering by feature, status, and date range.

Authorization: Required (Admin role)

20.7 Get AI Request Detail

> `GET /api/v1/admin/ai/requests/{id}`

Returns full detail for a single AI request including the user prompt and AI response excerpt.

Authorization: Required (Admin role)

20.8 List AI Feedback

> `GET /api/v1/admin/ai/feedback`

Returns user reactions (thumbs-up/thumbs-down) with triage status, user comments, and conversation context.

Authorization: Required (Admin role)

20.9 Update AI Feedback Status

> `PATCH /api/v1/admin/ai/feedback/{id}` — `application/json`

Updates the triage status, category classification, and admin notes for a feedback entry.

Authorization: Required (Admin role)

Request Body:

```
{
  "triageStatus": "Resolved",
  "category": "InaccurateInformation",
  "adminNotes": "Verified and added to training data exclusion list."
}
```

20.10 Get AI Performance Analytics

```
GET /api/v1/admin/ai/performance?days={days}
```

Returns day-by-day performance statistics including request counts, failure rates, latency percentiles, and feature breakdown.

Authorization: Required (Admin role)

Query Parameters:

Parameter	Type	Default	Description
`days`	`int`	`30`	Lookback window in days

20.11 Get AI Insights

```
GET /api/v1/admin/ai/insights
```

Returns AI-derived insights over the last 30 days: most-used feature, highest error feature, slowest/fastest features, and most-reported feedback category.

Authorization: Required (Admin role)

20.12 Get Usage Intelligence Overview

```
GET /api/v1/admin/usage-intelligence/overview
```

Returns a high-level summary of AI usage patterns, flagged request counts, users requiring review, and warnings sent.

Authorization: Required (Admin role)

20.13 Get User Attention Queue

```
GET /api/v1/admin/usage-intelligence/attention-queue
```

Returns a paginated list of users who require administrative attention based on flagged AI usage patterns.

Authorization: Required (Admin role)

20.14 Get Usage User Detail

```
GET /api/v1/admin/usage-intelligence/users/{userId}
```

Returns the full usage profile for a specific user: total requests, flagged items, warnings sent, AI restriction status, and complete action history.

Authorization: Required (Admin role)

20.15 Take Usage Action

> `POST /api/v1/admin/usage-intelligence/users/{userId}/actions` — `application/json`

Records an administrative action against a user (e.g., issue warning, restrict AI access, suspend account).

Authorization: Required (Admin role)

Request Body:

```
{
  "actionType": "RestrictAiAccess",
  "notes": "Repeated policy-violating queries. Restricted for 7 days."
}
```

Admin Review Management

20.16 Get All Reviews (Admin)

> `GET /api/reviews/admin`

Returns all reviews with full filtering options including status, category, star range, and sort order.

Authorization: Required (Admin role)

20.17 Reply to Review

> `POST /api/reviews/{id}/reply` — `application/json`

Posts an official admin response to a user review.

Authorization: Required (Admin role)

20.18 Toggle Review Visibility

> `PATCH /api/reviews/{id}/visibility` — `application/json`

Shows or hides a review from the public listing.

Authorization: Required (Admin role)

20.19 Update Review Status / Category

> `PATCH /api/reviews/{id}/status` / `PATCH /api/reviews/{id}/category`

Updates the moderation status or category classification of a review.

Authorization: Required (Admin role)

20.20 Update Admin Notes on Review

> `PATCH /api/reviews/{id}/notes`

Attaches internal admin notes to a review (not visible to the submitting user).

Authorization: Required (Admin role)

20.21 Delete Review

> `DELETE /api/reviews/{id}`

Permanently deletes a review.

Authorization: Required (Admin role)

21. Data Models Reference

AuthResponseDto

Property	Type	Description
`accessToken`	`string`	JWT Bearer token
`refreshToken`	`string`	Refresh token for session renewal
`accessTokenExpiresAt`	`datetime`	UTC expiry of access token
`refreshTokenExpiresAt`	`datetime`	UTC expiry of refresh token
`hasEmergencyContacts`	`boolean`	Whether the user has emergency contacts
`profileImageUrl`	`string?`	Profile picture URL

AccountDto

Property	Type	Description
`userId`	`guid`	Unique user identifier
`firstName`	`string`	Given name
`lastName`	`string`	Family name
`email`	`string`	Email address
`phoneNumber`	`string`	Mobile phone number
`phoneNumberConfirmed`	`boolean`	Whether the phone number is verified
`emailConfirmed`	`boolean`	Whether the email is verified
`role`	`string`	`"User"` or `"Admin"`
`profileImageUrl`	`string?`	Profile picture URL

PatientProfileDto

Property	Type	Description
`id`	`guid`	Profile unique identifier
`userId`	`guid?`	Linked user account ID (null for managed profiles)
`firstName`	`string`	Patient's first name
`lastName`	`string`	Patient's last name
`dateOfBirth`	`DateOnly`	Date of birth
`gender`	`string`	`"Male"` or `"Female"`
`bloodType`	`string?`	Blood type string (e.g., `"A+"`)
`height`	`decimal?`	Height in cm
`weight`	`decimal?`	Weight in kg
`profileImageUrl`	`string?`	Profile photo URL
`allergies`	`AllergyDto[]`	List of known allergies
`chronicDiseases`	`ChronicDiseaseDto[]`	List of chronic conditions
`createdAt`	`datetime`	Profile creation timestamp
`updatedAt`	`datetime?`	Last modification timestamp

AllergyDto

Property	Type	Description
`id`	`guid`	Allergy unique identifier
`name`	`string`	Allergen name
`severity`	`string`	`"Low"`, `"Medium"`, or `"High"`

ChronicDiseaseDto

Property	Type	Description
`id`	`guid`	Chronic disease record identifier
`name`	`string`	Condition name
`status`	`string`	`"Active"`, `"Managed"`, or `"Resolved"`
`diagnosedAt`	`DateOnly?`	Diagnosis date

EmergencyContactResponseDto

Property	Type	Description
`id`	`guid`	Contact identifier
`userId`	`guid`	Owning user's ID
`name`	`string`	Contact's full name
`phoneNumber`	`string`	Contact's phone number
`relation`	`string`	Relationship to the patient

AppointmentResponseDto

Property	Type	Description
`id`	`guid`	Appointment identifier
`appointmentType`	`int`	Appointment type enum value
`customType`	`string?`	Custom type label when `appointmentType` = Other
`title`	`string`	Appointment title
`provider`	`string`	Doctor or clinic name
`appointmentDateTime`	`datetime`	Scheduled UTC datetime
`reminderOffsetMinutes`	`int?`	Reminder lead time in minutes
`notes`	`string?`	Optional notes
`status`	`string`	`"Scheduled"`, `"Completed"`, or `"Cancelled"`
`createdAt`	`datetime`	Record creation timestamp
`updatedAt`	`datetime?`	Last modification timestamp

LabReportResponseDto

Property	Type	Description
`id`	`guid`	Report identifier
`labName`	`string`	Laboratory name
`doctorName`	`string?`	Requesting physician
`reportDate`	`string?`	Report date string
`ocrText`	`string?`	Raw OCR text extracted from the image
`summary`	`string?`	AI-generated patient-friendly explanation
`imageUrl`	`string?`	Stored image URL
`createdAt`	`datetime`	Record creation timestamp
`results`	`LabResultResponseDto[]`	Individual test results

LabResultResponseDto

Property	Type	Description
`id`	`guid`	Result identifier
`testName`	`string`	Test name (e.g., "Hemoglobin")
`value`	`string?`	Measured value
`unit`	`string?`	Unit of measurement
`normalRange`	`string?`	Reference range
`status`	`string?`	`"Normal"`, `"Low"`, or `"High"`

ImagingReportResponseDto

Property	Type	Description
`id`	`guid`	Report identifier
`imagingType`	`string`	Modality (e.g., "MRI", "CT", "X-Ray")
`bodyPart`	`string`	Body region examined
`findings`	`string`	Radiologist's findings
`impression`	`string`	Clinical impression / conclusion
`doctorName`	`string?`	Ordering or reporting physician
`reportDate`	`string?`	Report date string
`imageUrl`	`string?`	Stored image URL
`ocrText`	`string?`	Raw OCR text
`summary`	`string?`	AI-generated plain-language explanation
`createdAt`	`datetime`	Record creation timestamp

GeneralDocumentResponseDto

Property	Type	Description
`id`	`guid`	Document identifier
`title`	`string`	Document title
`description`	`string`	User description
`aiSummary`	`string?`	AI-generated summary
`imagePath`	`string?`	Stored image URL
`documentType`	`string?`	Classification (e.g., "DischargeSummary")
`doctorName`	`string?`	Associated physician
`hospitalOrClinic`	`string?`	Issuing institution
`documentDate`	`string?`	Document date string
`ocrText`	`string?`	Raw OCR text
`createdAt`	`datetime`	Record creation timestamp
`analysisStatus`	`string`	`"Pending"`, `"Processing"`, `"Completed"`, or `"Failed"`
`failureReason`	`string?`	AI failure reason if status is `"Failed"`

PrescriptionResponseDto

Property	Type	Description
`id`	`guid`	Prescription identifier
`doctorName`	`string`	Prescribing physician
`patientName`	`string`	Patient name on prescription
`prescriptionDate`	`string?`	Prescription issue date
`imagePath`	`string?`	Stored image URL
`createdAt`	`datetime`	Record creation timestamp
`medicines`	`PrescriptionMedicineResponseDto[]`	List of prescribed medicines

PrescriptionMedicineResponseDto

Property	Type	Description
`id`	`guid`	Medicine item identifier
`medicineName`	`string`	Medicine name
`dosage`	`string?`	Dosage instructions
`frequency`	`string?`	Dosing frequency
`duration`	`string?`	Treatment duration
`notes`	`string?`	Additional instructions

UserMedicineResponseDto

Property	Type	Description
`id`	`guid`	Medicine identifier
`medicineName`	`string`	Medicine name
`dosage`	`string`	Dosage
`frequency`	`string`	Administration frequency
`duration`	`string`	Treatment duration
`notes`	`string?`	Optional notes
`imagePath`	`string?`	Medicine box image URL
`source`	`string`	`"Manual"`, `"Prescription"`, or `"MedicineBox"`
`createdAt`	`datetime`	Record creation timestamp
`updatedAt`	`datetime?`	Last modification timestamp

MedicineReminderResponseDto

Property	Type	Description
`id`	`guid`	Reminder schedule identifier
`userMedicineId`	`guid`	Associated medicine
`reminderTime`	`TimeSpan`	Clock time (`HH:mm:ss`)
`startDate`	`DateOnly`	First active day
`endDate`	`DateOnly`	Last active day
`repeatType`	`string`	`"Once"` , `"Daily"` , `"Weekly"` , or `"Monthly"`
`isEnabled`	`boolean`	Whether the reminder is currently active
`lastTriggeredAt`	`datetime?`	Last time this reminder fired
`createdAt`	`datetime`	Record creation timestamp
`updatedAt`	`datetime?`	Last modification timestamp

MedicationReminderLogDto

Property	Type	Description
`id`	`guid`	Log entry identifier
`medicineReminderId`	`guid`	Source reminder schedule
`userHealthProfileId`	`guid`	Associated health profile
`medicineName`	`string`	Medicine name
`dosage`	`string`	Dosage
`reminderTime`	`TimeSpan`	Configured reminder clock time
`scheduledDate`	`DateOnly`	The date this log is for
`scheduledTime`	`TimeSpan`	The actual scheduled time for this attempt
`reminderNumber`	`int`	Attempt number (1, 2, or 3)
`status`	`string`	`"Pending"`, `"Confirmed"`, `"Skipped"`, or `"Snoozed"`
`isActionable`	`boolean`	True for exactly one log per medicine per day at any given time
`sentAt`	`datetime?`	When the push notification was sent
`confirmedAt`	`datetime?`	When the user confirmed intake
`createdAt`	`datetime`	Log entry creation timestamp

ConversationSummaryDto

Property	Type	Description
`id`	`guid`	Conversation identifier
`userHealthProfileId`	`guid`	Associated health profile
`title`	`string`	Conversation title
`lastMessageAt`	`datetime?`	Timestamp of the most recent message
`createdAt`	`datetime`	Conversation creation timestamp

UserNotificationDto

Property	Type	Description
`id`	`guid`	Notification identifier
`title`	`string`	English title
`body`	`string`	English body text
`titleAr`	`string?`	Arabic title
`bodyAr`	`string?`	Arabic body text
`type`	`string`	Notification category (e.g., `"MedicationReminder"`, `"Invitation"`)
`isRead`	`boolean`	Whether the notification has been read
`readAt`	`datetime?`	When the notification was marked as read
`createdAt`	`datetime`	Notification creation timestamp

22. Error Reference

All error responses follow the standard envelope with `success: false`:

```
{
  "success": false,
  "message": "Validation failed.",
  "data": null,
  "errors": [
    "Email address is required.",
    "Password must contain at least one uppercase letter."
  ],
  "errorCode": "VALIDATION_ERROR"
}
```

Common Error Codes

Status	Scenario	Example Message
`400`	Validation failure	`"Password must be at least 8 characters."`
`400`	Business rule violation	`"Appointment date must be in the future."`
`400`	Token expired or invalid	`"Refresh token has expired."`
`401`	No or invalid Bearer token	`"Authentication required."`
`401`	Account deactivated	`"Your account has been deactivated."`
`403`	Insufficient role	`"You do not have permission to perform this action."`
`403`	Profile access denied	`"You do not have access to this health profile."`
`404`	Resource not found	`"Patient profile not found."`
`409`	Duplicate registration	`"An account with this email address already exists."`
`500`	Unhandled exception	`"An unexpected error occurred. Please try again."`

23. Authorization Matrix

Endpoint Group	Anonymous	User (no profile)	Profile Owner	Profile Manager	Profile Viewer	Admin
Register / Login / Google	✗	✓	✓	✓	✓	✓
Public Reviews	✗	✓				✓
My Account (CRUD)	✗	✗	✓	✓	✓	✓
Emergency Contacts (CRUD)	✗	✗	✓	✓	✗	✓
Create Patient Profile			N/A	N/A	N/A	
View Profile	✗	✗	✓	✗	✗	✓
Edit Profile (demographics, allergies, diseases)	✗	✗	✓	✗	✗	✓
Delete Profile						
Invite / Manage Members	✗	✗	✓	✓	✓	✓
Change Member Role / Revoke	✗	✗	✓	✓	✗	✓
Lab / Imaging / Documents (read)	✗	✗	✓	✓	✓	✓
Lab / Imaging / Documents (write)	✗	✗	✓	✓	✗	✓
Prescriptions (read)	✗	✗	✓	✓	✗	✓
Prescriptions (write)	✗	✗	✓	✓	✓	✓
Medicines (read)	✗	✗	✓	✓	✗	✓
Medicines (write)	✗	✗	✓	✓	✓	✓
Appointments (read)	✗	✗	✓	✓	✗	✓
Appointments (write)	✗	✗	✓	✓	✗	✓
Medication Reminders	✗	✗	✓	✓	✓	✓
AI Chat	✗	✓	✓	✓	✓	✓
Voice Agent						
Medical Report PDF						
Notifications						

Endpoint Group	Anonymous	User (no profile)	Profile Owner	Profile Manager	Profile Viewer	Admin
Submit Review	✗	✗	✗	✗	✗	✓
Admin Dashboard / Users	✗	✗	✗	✗	✗	✓
Admin AI Management	✗	✗	✗	✗	✗	✓
Admin Reviews Management	✗	✗	✗	✗	✗	✓

24. Request Lifecycle

Every request to the Rafiq API follows this processing pipeline:



For AI-powered operations, the handler additionally calls the AWS Bedrock AI service (Claude model) for document extraction, health summaries, or chat responses. Async operations return immediately and deliver results via SignalR.

25. Best Practices

Pagination

All list endpoints that may return large datasets support pagination. Always specify `page` and `pageSize` parameters. The default page size is typically 20–30 items. Use the `TotalPages` and `TotalCount` fields from `PagedResult<T>` responses to implement navigation controls.

Token Management

Store the `accessToken` in memory (not `localStorage`) when possible for security. Store the `refreshToken` in a secure `httpOnly` cookie or encrypted local storage. Always check token expiry (`accessTokenExpiresAt`) before making requests and proactively refresh rather than waiting for a `401`.

File Uploads

- Maximum request size for image uploads is **6 MB**.
- Supported image types: JPEG, PNG, WEBP, GIF.
- For large documents, prefer the **async upload** endpoint (`/upload-async`) and use SignalR or polling to track completion.
- Always upload the image first and use the returned URL when saving the record.

Error Handling

Always check the `success` field before consuming `data`. Parse the `errors` array for user-facing validation messages. Use `errorCode` for programmatic branching when available.

Real-Time Updates (SignalR)

Connect to the SignalR hub at `/hubs/notifications` using a valid Bearer token. Subscribe to: - `ChatResponse` — AI chat assistant replies - `VoiceAgentResponse` / `VoiceAgentThinking` / `VoiceAgentError` — Voice agent events - `DocumentAnalysisCompleted` — Async document processing completion - Push notification events — In-app notification delivery

Localization

AI Chat and Voice Agent both accept a `language` parameter (`"en"` or `"ar"`). Notification bodies are returned in both languages (`body` and `bodyAr`) to support multilingual clients.

Security

- Never expose API keys to the browser. Use the `/api/speech/token` and `/api/speech/elevenlabs-tts` proxy endpoints instead.

- Validate all profile access on the server side — the API enforces access control at the handler level, not just the controller.
- The AI access restriction system (`isAiRestricted`) is enforced server-side. Check `/api/chat/ai-status` before rendering the chat UI.

26. Glossary

Term	Definition
Health Profile	A patient-centric data container holding demographics, allergies, chronic diseases, and all associated medical records
Managed Profile	A health profile created by a user on behalf of a dependent (e.g., child, elderly parent) that is not linked to any user account
Access Role	The permission level of a user within a specific health profile: `Owner`, `Manager`, or `Viewer`
JWT	JSON Web Token — a signed, time-limited credential used to authenticate API requests
Refresh Token	A long-lived, single-use token exchanged for a new access token pair when the current access token expires
MediatR	A mediator library for .NET that decouples controllers from handlers using the Command/Query pattern
CQRS	Command Query Responsibility Segregation — separates read operations (Queries) from write operations (Commands)
OCR	Optical Character Recognition — the AI process of extracting text from uploaded document images
SignalR	Microsoft's real-time messaging library used to push AI responses, notifications, and document status updates to connected clients
Reminder Log	A runtime entry generated by the Medication Reminder Engine representing a specific dose occurrence for a patient on a given day
Actionable Reminder	The single reminder log entry for a given medicine and day that the patient should currently act on (confirm, skip, or snooze)
Triage Status	The administrative review state of a negative AI feedback entry: `New`, `Investigating`, `Resolved`, or `Ignored`
AI Restriction	An administrative flag on a user account that prevents them from sending messages to the AI chat or voice agent
Egyptian Mobile Number	A phone number matching the pattern `^01[0125][0-9]{8}$` (Vodafone: 010, Orange: 012, Etisalat: 011, WE: 015)
Bearer Token	The authentication scheme; the access token is sent as `Authorization: Bearer <token>`
Clean Architecture	An architectural pattern separating the codebase into Domain, Application, Infrastructure, and API layers to enforce dependency rules

This document covers API version 1.0. For questions, contact the Rafiq development team.